Lesson 09 — Jacobians and Differential Kinematics

Physical AI · Module 2: Robot Kinematics

Status: Practical experiment passed; final conceptual reinforcement pending.

Learning objectives

1. Explain why a Jacobian is useful for controlling robot motion.
2. Derive the Jacobian of a two-link planar robot from forward kinematics.
3. Calculate the end-effector velocity from joint velocities.
4. Recognize singular and near-singular configurations.

1. From forward kinematics to velocities

Forward kinematics finds position from joint angles:

$$x=L_1\cos\theta_1+L_2\cos(\theta_1+\theta_2),\quad y=L_1\sin\theta_1+L_2\sin(\theta_1+\theta_2).$$

The Jacobian maps instantaneous joint angular velocities (rad/s) to end-effector linear velocities (m/s):

$$\begin{bmatrix}\dot x\\\dot y\end{bmatrix}= J(\theta_1,\theta_2) \begin{bmatrix}\dot\theta_1\\\dot\theta_2\end{bmatrix}.$$

2. Deriving the Jacobian

Differentiate x and y with respect to each joint angle:

$$J=\begin{bmatrix} -L_1\sin\theta_1-L_2\sin(\theta_1+\theta_2)&-L_2\sin(\theta_1+\theta_2)\\ L_1\cos\theta_1+L_2\cos(\theta_1+\theta_2)&L_2\cos(\theta_1+\theta_2) \end{bmatrix}.$$

The angles in Python's NumPy trig functions must be in radians.

3. Worked numerical example

For L1 = 3 m, L2 = 2 m, shoulder = 0 degrees, elbow = 90 degrees:

$$J=\begin{bmatrix}-2&-2\\3&0\end{bmatrix}.$$

For joint velocities [0.1, 0.2] rad/s:

```
[\dot x,\dot y]^T=J[0.1,0.2]^T=[-0.6,0.3]^T\text{ m/s}.
```

Interpretation: the gripper initially moves to the left at 0.6 m/s and upward at 0.3 m/s.

4. Python experiment — Velocity Jacobian

Save as `lesson_09_jacobian.py`:

```
import numpy as np

L1, L2 = 3.0, 2.0
t1, t2 = np.deg2rad([0.0, 90.0])
J = np.array([
    [-L1*np.sin(t1)-L2*np.sin(t1+t2), -L2*np.sin(t1+t2)],
    [ L1*np.cos(t1)+L2*np.cos(t1+t2),  L2*np.cos(t1+t2)]
])
qdot = np.array([0.1, 0.2])
velocity = J @ qdot
assert np.allclose(velocity, [-0.6, 0.3], atol=1e-9)
print('Jacobian:', J)
print('Tip velocity:', velocity, 'm/s')
print('Verification: PASSED')
```

Recorded learner result: Experiment passed, and learner correctly identified the Jacobian as a velocity relationship.

5. Singularities: what they mean

For this two-link planar arm:

```
\det J=L_1L_2\sin\theta_2.
```

At theta2 = 0 degrees or 180 degrees, the determinant is zero and the Jacobian loses rank. With L1 = 3 and L2 = 2 at theta1 = theta2 = 0:

```
J=\begin{bmatrix}0&0\\5&2\end{bmatrix}.
```

This gives **xdot = 0** at that instant but **ydot = 5*theta1dot + 2*theta2dot**. The robot can still move its joints, and the tip can still move in some directions. A singularity means **not all instantaneous Cartesian velocity directions are achievable**, not that the joints are frozen or the motors are broken.

Near singularity, the inverse differential-kinematics problem may become ill-conditioned and require unusually large joint velocities for some desired tip velocities.

6. Python experiment — Singularity analysis

Save as `lesson_09_singularity.py`:

```
import numpy as np

L1, L2 = 3.0, 2.0

def jacobian(t1_deg, t2_deg):
```

```
    t1, t2 = np.deg2rad([t1_deg, t2_deg])
    return np.array([
        [-L1*np.sin(t1)-L2*np.sin(t1+t2), -L2*np.sin(t1+t2)],
        [ L1*np.cos(t1)+L2*np.cos(t1+t2),  L2*np.cos(t1+t2)]
    ])

for elbow in [90, 5, 0]:
    J = jacobian(0, elbow)
    print('Elbow:', elbow, 'degrees')
    print('det(J):', np.linalg.det(J))
    print('rank(J):', np.linalg.matrix_rank(J))

J0 = jacobian(0, 0)
assert np.allclose(J0, [[0, 0], [5, 2]])
assert np.linalg.matrix_rank(J0) == 1
print('Singularity verification: PASSED')
```

Expected comparison:

Elbow angle	Determinant	Rank	Singular?
90 degrees	6.000000	2	No
5 degrees	approx. 0.522934	2	No
0 degrees	0.000000	1	Yes

Recorded learner result: Singularity Python experiment passed.

7. Connection to Isaac Sim

The same L1 = 3 m and L2 = 2 m geometry was used in the two-link visual USD arm from the preceding IK exercise. Straightening the elbow to 0 degrees gives a singular configuration. For Lesson 9 no additional Isaac Sim experiment was conducted; this connection is conceptual. The cubes in the existing demonstration are visual primitives, not an articulated dynamic robot.

8. Assessment and next checkpoint

- Python velocity experiment: **Passed**.

- Jacobian conceptual question: **Passed**.

- Python singularity experiment: **Passed**.

- Final conceptual assessment: **Correction needed**. Learner answered that a singular arm cannot move any joint. Correct idea: it cannot generate arbitrary instantaneous end-effector velocities; joints can still move.

Final reinforcement prompt: At a fully extended configuration, can the arm's joints still rotate? What instantaneous gripper motion is restricted?

Lesson-completion rule: Mark this lesson completed after the learner correctly explains the singularity distinction. Then proceed to Lesson 10 in accelerated mode.

9. Key takeaways

- FK maps joint positions to tip positions; the Jacobian maps joint velocities to tip velocities.
- The Jacobian depends on the current joint configuration.
- A rank-deficient Jacobian limits instantaneous tip motion; it does not disable joint motion.
- A near-singular configuration may be numerically sensitive even if its determinant is not exactly zero.